CONSULTING REPORT - 2

Logical Design and Physical Design of the Data Warehouse Schema

Design and Implementation of a Data Warehouse for a Retail Store with Store-level Data

ISTM 637 - Data Warehousing

Texas A&M University

Spring 2026

Team 1:

Nisarg Sonar

Bhavik Dalal

Yifei Wang

Date: March 18, 2026

Table of Contents

1. Introduction 3

1.1 Hybrid Data Pipeline Overview 3

2. Overview of Kimball's Methodology 4

3. Data Warehouse Logical Design (Star Schema Design) 7

3.1 Applying Kimball's Methodology to DFF 7

3.2 Star Schema Table Definitions 11

3.3 Star Schema ERD 15

3.4 Schema Justification for Each Business Question 16

3.5 Independent Data Marts Supported by Schema 18

4. Develop Two Mapping Tables 18

4.1 Mapping Table #1: Source Files to Staging Tables 19

4.2 Mapping Table #2: Staging Tables to Data Mart Tables 21

5. Physical Design 23

1. Introduction

This report presents the logical design and physical design of the data warehouse for Dominick's Fine Foods (DFF). It builds on the requirements analysis, business questions, and preliminary schema proposed in Report 1.

Report 1 identified 10 business questions grounded in the DFF dataset and retail domain research, proposed a star schema, and confirmed that the schema can answer all 10 questions. In this report, we focus on the 5 business questions selected by the professor for implementation: BQ2, BQ3, BQ4, BQ8, and BQ9. These span four product categories (Soft Drinks, Canned Soup, Toothpaste, and Crackers) and exercise key OLAP operations such as roll-up, dice, NTILE, RANK, and LAG.

Design Specifications (from Report 1):

- Implementation Architecture: Hybrid Data Pipeline
- Warehouse Architecture: Independent Data Marts
- Modeling Scheme: Dimensional Modeling (Star Schema)
- OLAP Style: HOLAP (Hybrid Online Analytical Processing)
- Target Infrastructure: SQL Server 2016

1.1 Hybrid Data Pipeline Overview

Our warehouse follows a Hybrid Data Pipeline architecture combining batch ETL processing with a structured staging layer. The pipeline has three main stages:

Stage 1 - Data Extraction: Raw CSV files (Movement files, UPC files, DEMO.csv, CCOUNT.csv) are extracted from the DFF source system. These flat files serve as the operational data sources.

Stage 2 - Staging Area: Extracted data is loaded into relational staging tables (stg_Movement, stg_Product, stg_Store). During this stage, data cleansing is performed: null values are handled, invalid records (OK=0, PRICE=0) are filtered out, and transformations like category code derivation from filenames are applied. The staging area acts as a buffer between source and target.

Stage 3 - Presentation Server (Data Mart): Cleaned and transformed data is loaded from staging into the star schema data mart tables (FactWeeklySales, DimProduct, DimStore, DimTime, DimCategory, DimPromotion). Surrogate keys are generated, foreign key lookups are performed, and derived facts (revenue, unit_price, profit_margin_pct) are computed during this load. The data mart serves as the presentation server for OLAP queries.

The two mapping tables in Section 4 document the transformations at each stage: Mapping Table #1 covers Stage 1 to Stage 2, and Mapping Table #2 covers Stage 2 to Stage 3.

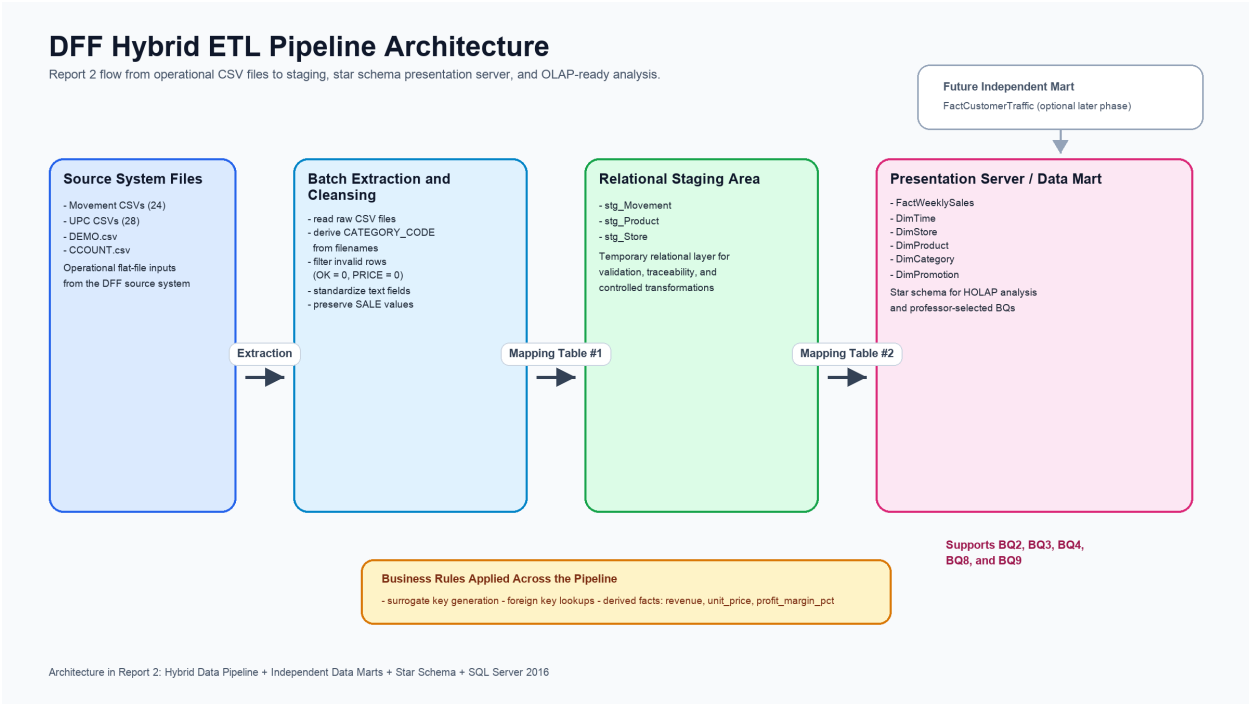

Figure 1. Hybrid ETL Pipeline Architecture for DFF

2. Overview of Kimball's Methodology

The data warehouse logical design follows Kimball's bottom-up methodology for building independent, conformable data marts. Dimensional modeling is a logical design technique that presents data in a standard format that is intuitive and allows for high-performance access. The fundamental idea is that every type of business data can be represented as a cube or hypercube. This approach is well suited for building independent data marts that can later be integrated through conformed dimensions.

Kimball's methodology consists of six steps. Each step builds on the previous one to progressively shape the data warehouse from business needs through logical design to physical implementation.

Step 1 - Requirements Analysis: The first step is to understand what the business needs from the data warehouse. This involves three activities: gathering business questions from stakeholders, looking for business objectives and issues that the warehouse should address, and collecting analysis requirements such as the types of reports, filters, and comparisons users need. Requirements analysis sets the foundation for all subsequent design decisions because the schema must be shaped by what users actually need to know, not by how data happens to be stored in the source system.

Step 2 - Build the Matrix: The matrix (also called the data mart/dimension matrix) is a grid that lists data marts along one axis and dimensions along the other, with intersections marked to show which dimensions each data mart uses. This step has three sub-steps: (2.1) list the data marts, starting with single-source data marts because they are simpler to implement and can deliver value to end users quickly; (2.2) list all the dimensions needed for filtering, grouping, and drilling into fact data; and (2.3) mark the intersections to identify conformed dimensions that are shared across multiple data marts. On average, each data mart uses about 10 dimensions, though this varies based on the complexity of the business domain.

Step 3 - Design Fact Tables: The fact table is the central table that stores the quantitative measurements (facts) of a business process. Designing it involves six sub-steps: (3A) choose the data mart to focus on, (3B) declare the grain, which means defining exactly what one row in the fact table represents, (3C) choose the facts and classify them as additive (can be summed across all dimensions), semi-additive (can be summed across some but not all dimensions), or non-additive (cannot be summed at all, such as ratios and percentages), (3D) include derived facts at design time rather than making end users calculate them, (3E) document all base and derived facts with clear definitions and formulas, and (3F) create a fact table diagram showing the fact table surrounded by its dimensions. A well-designed fact table should be deep (many rows) and narrow (few columns), containing mostly numeric measures and foreign keys.

Step 4 - Design Dimension Tables: Dimension tables store the descriptive, textual context that gives meaning to the numeric facts. Dimensions answer the "who, what, where, when, why, and

how" of business questions. Each dimension should document its explicit grain (what one row represents), approximate cardinality (how many rows it will have), and hierarchical relationships between attributes (for example, week rolls up to month, which rolls up to quarter, which rolls up to year). Surrogate keys, which are meaningless sequential integers, are used as primary keys instead of production keys or smart keys. Dimensions should be denormalized into flat, wide tables. Snowflaking (normalizing dimension attributes into separate tables) is not recommended because it slows browsing among attributes, confuses business users, increases the number of joins required, and raises administration cost.

Step 5 - Feedback for the Design: Before finalizing the schema, the design is validated through three rounds of feedback. Step 5A involves review by the IS (Information Systems) technical team to check whether any important source attributes are missing from the model. Step 5B involves core business users who verify that the schema supports the reporting, navigation, and drill-down operations they need (such as drill-down from year to quarter to month, slicing by a single dimension value, and dicing across multiple dimensions). Step 5C is a broader presentation to business users to validate the model against the full set of business questions and confirm that every question can be answered.

Step 6 - Data Sourcing: The final step identifies all data sources and plans how data will flow from source systems into the warehouse. Data sources are classified as formal (maintained by database staff in structured formats) or informal (personal databases, spreadsheets, or new data). For each source, a data source definition is created that includes the source name, platform, location, and description of contents. Data sourcing also establishes two types of mapping: Type 1 mapping defines how source data is extracted and loaded into staging tables, and Type 2 mapping defines how staging data is transformed and loaded into the presentation server (data mart) tables.

Why is this methodology important for independent data marts?

Kimball's methodology ensures that each data mart is driven by actual business needs rather than simply copying the structure of source systems. By using conformed dimensions (dimensions that are shared across data marts with consistent definitions), independent data marts can be queried together and integrated across the enterprise without building a monolithic enterprise data warehouse upfront. This means organizations can start small, deliver value quickly with a

focused data mart, and then expand incrementally by adding new data marts that share the same conformed dimensions. The emphasis on denormalized star schemas guarantees fast query performance and produces a structure that business users can understand without technical training, since each dimension corresponds to a familiar business concept.

3. Data Warehouse Logical Design (Star Schema Design)

This section presents the star schema design for DFF, created by following Kimball's six-step methodology in its entirety. We first walk through how each step was applied to arrive at our design, then present the resulting table definitions, ERD, and business question justifications.

3.1 Applying Kimball's Methodology to DFF

Step 1: Requirements Analysis for DFF

In Report 1, we studied the DFF dataset and identified 10 business questions from stakeholder analysis and retail domain research. The business objectives for DFF are: (1) understand weekly sales trends across product categories, (2) measure the effectiveness of different promotion types on sales volume, (3) compare store performance tiers using revenue and demographic data, and (4) track top-selling products and their week-over-week changes.

The analysis requirements include weekly sales trend tracking, promotional effectiveness measurement, store-level demographic segmentation, and product-level ranking with temporal comparisons. From the 10 questions, the professor selected the following 5 for this report:

BQ ID	Business Question	Category	OLAP Operation
BQ2	What were the total weekly unit sales of Soft Drinks across all stores for each week?	Soft Drinks (SDR)	Roll-up
BQ3	How do promotion weeks compare to non-promotion weeks in terms of sales volume?	Soft Drinks (SDR)	Dice
BQ4	Which promotion type (B=Bonus Buy, C=Coupon, S=Sale) generated the highest incremental unit sales lift in Canned Soup?	Canned Soup (CSO)	Dice
BQ8	Which stores fall into the top 25%, middle 50%, and bottom 25% of total revenue for Toothpaste, and how do their demographics differ?	Toothpaste (TPA)	NTILE
BQ9	For each week, rank the top 10 products (UPCs) in Crackers by unit sales and show their week-over-week	Crackers (CRA)	RANK+LAG

	sales change.		

Step 2: Building the Data Mart / Dimension Matrix

Step 2.1 - List the Data Marts: Following the principle of starting with single-source data marts, we identified FactWeeklySales as our primary data mart. It is sourced from the 24 Movement CSV files and captures weekly item-level sales across all DFF stores. This single data mart answers all 5 selected business questions, making it the core of our design.

Step 2.2 - List the Dimensions: Based on the filtering, grouping, and drilling requirements of the 5 BQs, we identified five dimensions:

- DimTime - provides week-level time hierarchy (week, month, quarter, year) for BQ2 weekly trending and BQ9 week-over-week comparisons
- DimStore - provides store attributes and demographics (income, education, poverty, urban status) for BQ8 store segmentation
- DimProduct - provides product attributes (UPC, description, size) for BQ9 product-level ranking
- DimCategory - provides product category filtering (SDR, CSO, TPA, CRA) needed by all 5 BQs to isolate specific categories
- DimPromotion - provides promotion type classification (No Promotion, Bonus Buy, Coupon, Sale) for BQ3 and BQ4

Step 2.3 - Mark the Intersections: Our design uses 5 dimensions for FactWeeklySales. While a typical enterprise has about 10 dimensions per data mart, the DFF dataset covers a single grocery retailer with a focused set of CSV files, not a multi-division corporation. The 5 dimensions are sufficient to support all the filtering, grouping, and drill operations needed by our 5 BQs.

Data Mart	DimTime	DimStore	DimProduct	DimCategory	DimPromotion	BQs Supported
FactWeeklySales	X	X	X	X	X	BQ2, BQ3, BQ4, BQ8, BQ9

All five dimensions are conformed, meaning they can be reused if additional data marts (for example, a customer traffic data mart built from CCOUNT.csv) are added in the future without any structural changes to the existing schema.

Step 3: Designing the Fact Table

Step 3A - Choose the Data Mart: FactWeeklySales is our primary data mart. It captures weekly item-level sales, which is the core business process for DFF and the process that all 5 BQs analyze.

Step 3B - Declare the Grain: Each individual fact record represents the weekly sales of one product (UPC) at one store. This is the finest grain available in the DFF Movement files. This grain supports all required aggregations: rolling up to weekly totals for BQ2, grouping by promotion for BQ3 and BQ4, summing revenue by store for BQ8, and ranking products within a week for BQ9.

Step 3C - Choose the Facts: Facts are classified by their additive properties:

- Additive (can be summed across all dimensions): units_sold (from MOVE), revenue (derived), gross_profit (from PROFIT)
- Non-additive (cannot be summed): unit_price (derived ratio), profit_margin_pct (derived ratio)
- Semi-additive (additive across some dimensions but not time): shelf_price (from PRICE), price_qty (from QTY)

The fact table has 13 columns (6 foreign keys + 7 measures), keeping it deep and narrow as recommended.

Step 3D - Include Derived Facts: Derived facts are included at design time rather than deferring them to end-user applications. Our derived measures and their calculations:

Derived Measure	Formula	Additivity	Purpose
revenue	units_sold x (shelf_price / price_qty)	Additive	BQ8 uses total revenue per store for quartile ranking
unit_price	shelf_price / price_qty	Non-additive	Per-unit price for pricing analysis
profit_margin_pct	(gross_profit / revenue) x 100	Non-additive	Measures profitability as a percentage

Revenue is additive because it is a dollar amount per transaction that can be summed across products, stores, and time. Unit_price and profit_margin_pct are ratios, and summing ratios produces meaningless results, which is why they are non-additive.

Step 3E - Document Base and Derived Facts: All base facts (units_sold, shelf_price, price_qty, gross_profit) and derived facts (revenue, unit_price, profit_margin_pct) are documented in the table definition in Section 3.2 and cross-referenced with the 5 BQs in Section 3.4.

Step 3F - Create Fact Table Diagram: A logical diagram of the fact table with all its dimensions is presented as the Star Schema ERD in Section 3.3. This diagram shows the context of FactWeeklySales and can be used for communication with end users, data access tool selection, and training.

Step 4: Designing the Dimension Tables

Each dimension table uses a surrogate key (a meaningless 4-byte integer) as its primary key. We avoid using smart keys or production keys. The original natural keys (STORE number, UPC code, WEEK number) are kept as regular attributes for traceability back to source data.

All dimensions are fully denormalized into flat, wide tables with no snowflaking. Snowflaking would slow browsing, cause user confusion, increase the number of joins needed for queries, and raise administration cost. For example, DimStore contains all 18 demographic attributes directly (income, education, poverty, urban status, etc.) rather than normalizing demographics into a separate table. This allows BQ8 to compare store demographics across revenue tiers with a single join.

The explicit grain and approximate cardinality of each dimension:

Dimension	Grain	Approx Rows	Key Attributes for BQs
DimTime	One row per unique week	~400	week_id, month, quarter, year
DimStore	One row per store	~107	store_id, avg_income, education_pct, poverty_pct, is_urban
DimProduct	One row per unique UPC	~14,000	upc, description, size
DimCategory	One row per product category	28	category_code (SDR, CSO, TPA, CRA)
DimPromotion	One row per promotion type	4	deal_code (NULL, B, C, S), is_promoted

Step 5: Feedback for the Design

Step 5A (IS Team): We verified that no important attributes are missing from either the fact table or the dimensions. All 10 original BQs from Report 1 are answerable with the current dimensional model. The fact table grain supports all required aggregations.

Step 5B (Core Business Users): The schema supports standard OLAP navigation: drill-down (expanding from year to quarter to month to week), drill-up (summarizing by collapsing hierarchies), slice (fixing one dimension value such as a specific category), dice (filtering across multiple dimensions), ranking (RANK and NTILE window functions), and windowed operations (LAG for week-over-week comparisons).

Step 5C (Business Users): All 5 selected BQs were tested against the schema. Each BQ is fully supported by the current fact table and dimension combination. Detailed justifications with data evidence are provided in Section 3.4.

Step 6: Data Sourcing

Our data sources are all formal, maintained as structured CSV files from the DFF dataset published by the Kilts Center at the University of Chicago Booth School of Business.

Source	Platform	Location	Description
Movement Files (24)	CSV	DFF data/Movement/	Weekly item-level sales: STORE, UPC, WEEK, MOVE, QTY, PRICE, SALE, PROFIT, OK
UPC Files (28)	CSV	DFF data/UPC/	Product master: COM_CODE, UPC, DESCRIP, SIZE, CASE, NITEM
DEMO.csv	CSV	DFF data/Demographics/	Store demographics: 107 stores, 18 demographic attributes per store
CCOUNT.csv	CSV	DFF data/Ccount/	Customer traffic counts by department per store (potential future data mart)

Data mapping follows two stages as prescribed by the methodology: Source to Staging (Type 1) and Staging to Presentation Server (Type 2). Both are documented in Section 4.

3.2 Star Schema Table Definitions

The following table definitions are the result of applying Kimball's six-step process. All tables use surrogate keys as primary keys.

FactWeeklySales (Fact Table)

Grain: One row per UPC x Store x Week. Source: 24 Movement CSV files. Estimated ~134.9M rows.

Column	Data Type	Description	Source	Additivity
sales_fact_id (PK)	INT	Surrogate key	Generated	-
product_key (FK)	INT	FK to DimProduct	UPC	-
store_key (FK)	INT	FK to DimStore	STORE	-
time_key (FK)	INT	FK to DimTime	WEEK	-
category_key (FK)	INT	FK to DimCategory	filename	-
promotion_key (FK)	INT	FK to DimPromotion	SALE	-
units_sold	INT	Units moved/sold	MOVE	Additive
unit_price	DECIMAL(8,2)	shelf_price / price_qty	Derived	Non-additive
shelf_price	DECIMAL(8,2)	Listed shelf price	PRICE	Semi-additive
price_qty	INT	Qty for listed price	QTY	Semi-additive
revenue	DECIMAL(12,2)	units_sold x unit_price	Derived	Additive
gross_profit	DECIMAL(10,2)	Gross profit	PROFIT	Additive
profit_margin_pct	DECIMAL(5,2)	(gross_profit/revenue) x 100	Derived	Non-additive

DimProduct

One row per unique UPC. ~14,000 rows. Source: 28 UPC CSV files.

Column	Data Type	Description	Source
product_key (PK)	INT	Surrogate key	Generated
upc (Unique)	BIGINT	Universal Product Code	UPC
description	VARCHAR(100)	Product description	DESCRIP
size	VARCHAR(20)	Package size	SIZE
case_pack	INT	Case pack quantity	CASE
commodity_code	INT	Manufacturer code	COM_CODE
item_number	BIGINT	Internal item number	NITEM

DimStore

One row per store. ~107 rows. Source: DEMO.csv. Fully denormalized.

Column	Data Type	Description	Source

store_key (PK)	INT	Surrogate key	Generated
store_id (Unique)	INT	Original store number	STORE
store_name	VARCHAR(50)	Store name	NAME
city	VARCHAR(40)	City	CITY
zip_code	VARCHAR(10)	ZIP code	ZIP
zone	INT	Pricing zone	ZONE
is_urban	BIT	Urban flag	URBAN
weekly_volume	INT	Avg weekly volume	WEEKVOL
avg_income	DECIMAL(10,2)	Avg area income	INCOME
education_pct	DECIMAL(5,2)	Pct higher education	EDUC
poverty_pct	DECIMAL(5,2)	Pct below poverty	POVERTY
avg_household_size	DECIMAL(4,2)	Avg household size	HSIZEAVG
ethnic_diversity	DECIMAL(5,2)	Ethnic diversity	ETHNIC
population_density	DECIMAL(10,2)	Population density	DENSITY
price_tier	VARCHAR(10)	Low/Medium/High	Derived: PRICLOW, PRICMED, PRICHIGH
age_under_9_pct	DECIMAL(5,2)	Pct under 9	AGE9
age_over_60_pct	DECIMAL(5,2)	Pct over 60	AGE60
working_women_pct	DECIMAL(5,2)	Pct working women	WORKWOM

DimTime

One row per unique week. ~400 rows. Derived from WEEK values (Week 1 = Sep 14, 1989).

Column	Data Type	Description	Source
time_key (PK)	INT	Surrogate key	Generated
week_id	INT	DFF week number	WEEK
week_start_date	DATE	Start date of the week	1989-09-14 + (week_id-1) x 7
week_end_date	DATE	End date of the week	week_start_date + 6
month	INT	Month (1 to 12)	Derived
month_name	VARCHAR(10)	Month name	Derived
quarter	INT	Quarter (1 to 4)	Derived
year	INT	Calendar year	Derived
fiscal_year	INT	DFF fiscal year	Derived
is_holiday_week	BIT	Holiday week flag	Engineered

DimCategory

One row per product category. 28 rows. Derived from source filenames.

Column	Data Type	Description	Source
category_key (PK)	INT	Surrogate key	Generated
category_code (Unique)	CHAR(3)	3-letter category code	filename
category_name	VARCHAR(30)	Full category name	Codebook
department	VARCHAR(20)	Department grouping	Engineered

Report 2 categories: SDR (Soft Drinks), CSO (Canned Soup), TPA (Toothpaste), CRA (Crackers).

DimPromotion

One row per promotion type. 4 rows. Derived from the SALE column in Movement files.

Column	Data Type	Description	Source
promotion_key (PK)	INT	Surrogate key	Generated
deal_code	CHAR(1)	Deal flag value	SALE
deal_type	VARCHAR(20)	Descriptive promotion name	Mapped from deal_code
is_promoted	BIT	Promoted yes/no flag	Derived

deal_code	deal_type	is_promoted
NULL or blank	No Promotion	0
B	Bonus Buy	1
C	Coupon	1
S	Sale/Discount	1

During ETL (Mapping Table #2), any Movement record where SALE is NULL or blank is mapped to the "No Promotion" row (promotion_key = 1, deal_code = NULL, is_promoted = 0). Records with SALE = B, C, or S are mapped to the corresponding promotion row.

3.3 Star Schema ERD

The Star Schema ERD below represents the logical design of our data warehouse. It shows FactWeeklySales at the center surrounded by the five conformed dimension tables. This diagram follows the fact table diagram approach from Step 3F, showing the context of the fact table with respect to all its dimensions.

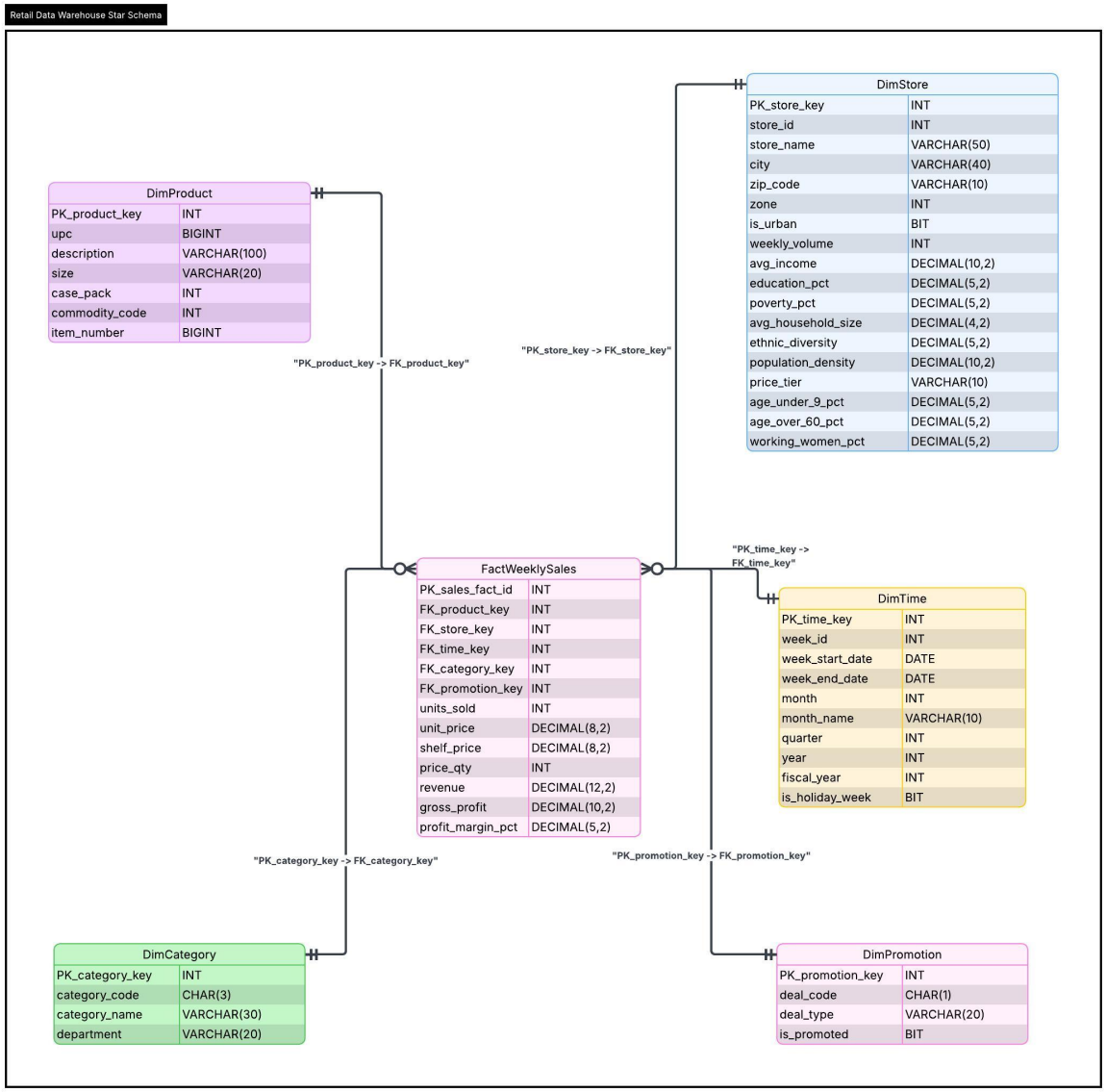

Figure 2. Star Schema ERD for DFF Data Warehouse

The ERD uses crow's foot notation to show 1:N relationships from each dimension to the fact table. FactWeeklySales holds five foreign keys (product_key, store_key, time_key,

category_key, promotion_key), each pointing to its corresponding dimension. This is a pure star schema with no inter-dimension foreign keys.

3.4 Schema Justification for Each Business Question

Each business question is justified below with the tables and columns required, the OLAP operation involved, and data evidence from Report 1 showing that the schema produces meaningful results.

BQ2: Total weekly unit sales of Soft Drinks across all stores

Tables Required: FactWeeklySales, DimTime, DimCategory

OLAP Operation: Roll-up (aggregate individual product-store-week records up to weekly totals)

The units_sold measure is fully additive. By filtering on DimCategory.category_code = 'SDR' and grouping by DimTime.week_id, the schema aggregates individual product-store records into total weekly unit sales across all stores. This is a classic roll-up operation.

Data Evidence (Report 1): Soft Drinks had the highest total unit volume with 29,222 units in a 1,000-row sample. Weekly trends showed baseline demand between 100,000 and 200,000 units with promotional spikes above 400,000 units.

BQ3: Promotion weeks vs non-promotion weeks in sales volume

Tables Required: FactWeeklySales, DimPromotion, DimCategory

OLAP Operation: Dice (filter by category and group by promotion status simultaneously)

By joining FactWeeklySales with DimPromotion on promotion_key, users can group units_sold by is_promoted (0 or 1) or by deal_type for further drill-down. DimPromotion cleanly separates promoted from non-promoted records. This is a dice operation because we filter on two dimensions at once (category and promotion).

Data Evidence (Report 1): Promoted Soft Drink items averaged 56.61 units compared to 20.27 for non-promoted, a 2.8x lift. Sale generated 102.97 avg units (about 7.2x lift), Bonus Buy 94.50 (about 6.6x), Coupon 60.51 (about 4.2x).

BQ4: Highest incremental unit sales lift by promotion type in Canned Soup

Tables Required: FactWeeklySales, DimPromotion, DimCategory

OLAP Operation: Dice (filter category + promotion type, compare to baseline)

The schema captures each record's promotion type through promotion_key pointing to DimPromotion.deal_code. The incremental lift is calculated by comparing the average units_sold during promoted periods (B, C, or S) against the average during non-promoted periods. Grouping by deal_code reveals which promotion type delivers the highest lift for Canned Soup.

Data Evidence (Report 1): In Canned Soup, the non-promotional baseline was 10.17 units. Bonus Buy added +10.86 units, Sale added +74.30 units. Sale was clearly the strongest promotion type for this category.

BQ8: Store quartile tiers by Toothpaste revenue with demographics

Tables Required: FactWeeklySales, DimStore, DimCategory

OLAP Operation: NTILE (ranking into quartiles) + Drill-down (demographics)

The revenue derived measure is summed by store_key while filtering DimCategory.category_code = 'TPA'. NTILE(4) then divides stores into four quartiles. Because DimStore is fully denormalized with all demographic attributes (income, education, poverty, urban status, household size, ethnic diversity, population density), the quartile results can be directly compared by demographics without any additional joins or snowflake lookups.

Data Evidence (Report 1): Using percentile thresholds (Q1=$39,020.77, Q3=$72,517.15), Top 25% stores averaged weekly customer volume of 565.04 compared to 347.06 for Bottom 25%. Among Top 25% stores, 73.9% were urban, indicating a strong demographic correlation with revenue.

BQ9: Weekly top 10 Cracker products with week-over-week change

Tables Required: FactWeeklySales, DimProduct, DimTime, DimCategory

OLAP Operation: RANK (ranking within each week) + LAG (comparing to previous week)

FactWeeklySales stores units_sold at the UPC x Store x Week grain. By aggregating by product_key and time_key while filtering DimCategory.category_code = 'CRA', we get weekly totals per product. RANK() OVER(PARTITION BY week_id) identifies the top 10 products per week, and LAG() computes the week-over-week change by comparing each product's current week units to its previous week units.

Data Evidence (Report 1): The WEEK, UPC, and MOVE fields directly support this query. Top 10 filtering and difference-from-previous calculations were demonstrated using PivotTables in Report 1.

3.5 Independent Data Marts Supported by Schema

This report implements one independent data mart: FactWeeklySales, which focuses on the weekly retail sales process and answers all 5 selected business questions. The schema is designed following the independent data marts architecture specified in the project requirements, where each data mart is a self-contained star schema built around a single business process.

Key characteristics of this architecture as applied to DFF:

- FactWeeklySales is our first and currently only data mart. It is fully self-contained, with its own fact table and five conformed dimensions.
- All five dimensions (DimTime, DimStore, DimProduct, DimCategory, DimPromotion) are conformed, meaning they use consistent keys and definitions. If a second data mart is added in the future, it can reuse these same dimensions without any changes.
- The DFF dataset provides a clear candidate for a second data mart: a FactCustomerTraffic mart built from CCOUNT.csv, sharing DimStore and DimTime with the existing FactWeeklySales mart.
- No centralized enterprise data warehouse is built upfront. Instead, we start with the most critical business process (weekly sales) and can expand incrementally as new analytical needs arise.

The star schema design ensures efficient querying through simple joins between the fact table and its dimensions. Business users can understand the structure naturally because each dimension represents a familiar business concept: time, store, product, category, and promotion.

4. Develop Two Mapping Tables

Mapping tables are a critical part of the ETL process. They document exactly how data flows from source systems through the staging area and into the final data mart tables. Without mapping tables:

- Developers would not know which source columns feed which target columns
- Data transformations (cleansing, derivation, type conversion) would be undocumented
- Future maintenance and troubleshooting would be difficult because the data lineage is lost
- Auditing and data governance would be impossible since there is no traceability

There are two types of mapping in Kimball's methodology: Source to Staging (Type 1) and Staging to Presentation Server (Type 2). These correspond to the two stages of our Hybrid Data Pipeline described in Section 1.1.

4.1 Mapping Table #1: Source Files to Staging Tables

This mapping defines how raw source data is extracted from CSV files and loaded into relational staging tables. The staging tables serve as a buffer where data cleansing and initial transformations are performed before loading into the data mart.

Raw DFF CSV files are not loaded directly into the data mart because they contain invalid records, inconsistent formatting, and no category identifiers. The staging layer acts as a buffer in the Hybrid Data Pipeline where data quality issues are resolved before any star schema loading occurs. During staging, rows with OK = 0 or PRICE = 0 are filtered out as unusable, product descriptions are cleaned of leading special characters (# and ~), and a CATEGORY_CODE column is derived from the source filename (for example, wsdr.csv produces the code 'SDR' for Soft Drinks). The tables below show only the four categories relevant to the selected business questions (SDR, CSO, TPA, CRA), but all 24 Movement files and 28 UPC files follow the same extraction pattern.

Movement Files to stg_Movement:

Source File Name	Source File Attribute	Mapping	Staging Table Type	Staging Table Name	Staging Table Attribute
wsdr.csv	STORE	Copy	Relation	stg_Movement	STORE
wsdr.csv	UPC	Copy	Relation	stg_Movement	UPC
wsdr.csv	WEEK	Copy	Relation	stg_Movement	WEEK
wsdr.csv	MOVE	Copy	Relation	stg_Movement	MOVE
wsdr.csv	QTY	Copy	Relation	stg_Movement	QTY
wsdr.csv	PRICE	Copy	Relation	stg_Movement	PRICE
wsdr.csv	SALE	Copy (keep NULL as-is)	Relation	stg_Movement	SALE
wsdr.csv	PROFIT	Copy	Relation	stg_Movement	PROFIT
wsdr.csv	OK	Copy	Relation	stg_Movement	OK
wsdr.csv	(filename)	Extract 'SDR'	Relation	stg_Movement	CATEGORY_CODE
WCSO-Done.csv	(all cols)	Same as above	Relation	stg_Movement	(same)

WCSO-Done.csv	(filename)	Extract 'CSO'	Relation	stg_Movement	CATEGORY_CODE
WTPA_done.csv	(all cols)	Same as above	Relation	stg_Movement	(same)
WTPA_done.csv	(filename)	Extract 'TPA'	Relation	stg_Movement	CATEGORY_CODE
Done-WCRA.csv	(all cols)	Same as above	Relation	stg_Movement	(same)
Done-WCRA.csv	(filename)	Extract 'CRA'	Relation	stg_Movement	CATEGORY_CODE

Note: All 24 Movement files follow this same pattern. Only the 4 categories relevant to the selected BQs are shown. During staging, rows with OK = 0 or PRICE = 0 are filtered out as invalid records.

UPC Files to stg_Product:

Source File Name	Source File Attribute	Mapping	Staging Table Type	Staging Table Name	Staging Table Attribute
UPCSDR.csv	COM_CODE	Copy	Relation	stg_Product	COM_CODE
UPCSDR.csv	UPC	Copy	Relation	stg_Product	UPC
UPCSDR.csv	DESCRIP	Transform: Strip #/~	Relation	stg_Product	DESCRIP
UPCSDR.csv	SIZE	Transform: Standardize	Relation	stg_Product	SIZE
UPCSDR.csv	CASE	Copy	Relation	stg_Product	CASE_PACK
UPCSDR.csv	NITEM	Copy	Relation	stg_Product	NITEM
UPCSDR.csv	(filename)	Derive 'SDR'	Relation	stg_Product	CATEGORY_CODE
UPCCSO.csv	(all cols)	Same; CODE='CSO'	Relation	stg_Product	(same)
UPCTPA.csv	(all cols)	Same; CODE='TPA'	Relation	stg_Product	(same)
UPCCRA.csv	(all cols)	Same; CODE='CRA'	Relation	stg_Product	(same)

Demographics (DEMO.csv) to stg_Store:

Source File Name	Source File Attribute	Mapping	Staging Table Type	Staging Table Name	Staging Table Attribute
DEMO.csv	STORE	Copy	Relation	stg_Store	STORE
DEMO.csv	NAME	Copy (NULL to UNKNOWN)	Relation	stg_Store	NAME

DEMO.csv	CITY	Copy (NULL to UNKNOWN)	Relation	stg_Store	CITY
DEMO.csv	ZIP	Copy	Relation	stg_Store	ZIP
DEMO.csv	ZONE	Copy	Relation	stg_Store	ZONE
DEMO.csv	URBAN	Copy	Relation	stg_Store	URBAN
DEMO.csv	WEEKVOL	Copy	Relation	stg_Store	WEEKVOL
DEMO.csv	INCOME	Copy	Relation	stg_Store	INCOME
DEMO.csv	EDUC	Copy	Relation	stg_Store	EDUC
DEMO.csv	POVERTY	Copy	Relation	stg_Store	POVERTY
DEMO.csv	HSIZEAVG	Copy	Relation	stg_Store	HSIZEAVG
DEMO.csv	ETHNIC	Copy	Relation	stg_Store	ETHNIC
DEMO.csv	DENSITY	Copy	Relation	stg_Store	DENSITY
DEMO.csv	AGE9	Copy	Relation	stg_Store	AGE9
DEMO.csv	AGE60	Copy	Relation	stg_Store	AGE60
DEMO.csv	WORKWOM	Copy	Relation	stg_Store	WORKWOM
DEMO.csv	PRICLOW,PRICMED,PRICHIGH	Derive: flag=1 to Low/Med/High	Relation	stg_Store	PRICE_TIER

4.2 Mapping Table #2: Staging Tables to Data Mart Tables

This mapping defines how staging data is transformed and loaded into the data mart (presentation server) tables. This is the second stage of the Hybrid Data Pipeline where surrogate keys are generated, foreign key lookups are performed, and derived facts are computed.

Once the staging tables are clean, the data is loaded into the star schema on the presentation server. During this stage, integer surrogate keys are generated for every dimension and fact row, and foreign key lookups connect each staging record to the correct conformed dimension (for example, UPC values in stg_Movement are matched to product_key in DimProduct). Derived facts are also computed at this point: unit_price is calculated as shelf_price divided by price_qty, revenue as units_sold multiplied by unit_price, and profit_margin_pct as gross_profit divided by revenue times 100. The result is a set of OLAP-ready fact and dimension tables that directly support BQ2, BQ3, BQ4, BQ8, and BQ9.

Staging Table	Staging Table Attribute	Mapping	Data Mart Table Type	Data Mart Table Name	Data Mart Table Attribute
stg_Store	(auto)	Generate surrogate key	Dimension	DimStore	store_key (PK)
stg_Store	STORE	Copy	Dimension	DimStore	store_id

stg_Store	NAME	Copy	Dimension	DimStore	store_name
stg_Store	CITY	Copy	Dimension	DimStore	city
stg_Store	ZIP	Copy	Dimension	DimStore	zip_code
stg_Store	ZONE	Copy	Dimension	DimStore	zone
stg_Store	URBAN	Copy	Dimension	DimStore	is_urban
stg_Store	WEEKVOL	Copy	Dimension	DimStore	weekly_volume
stg_Store	INCOME	Copy	Dimension	DimStore	avg_income
stg_Store	EDUC	Copy	Dimension	DimStore	education_pct
stg_Store	POVERTY	Copy	Dimension	DimStore	poverty_pct
stg_Store	HSIZEAVG	Copy	Dimension	DimStore	avg_household_size
stg_Store	ETHNIC	Copy	Dimension	DimStore	ethnic_diversity
stg_Store	DENSITY	Copy	Dimension	DimStore	population_density
stg_Store	PRICE_TIER	Copy	Dimension	DimStore	price_tier
stg_Store	AGE9	Copy	Dimension	DimStore	age_under_9_pct
stg_Store	AGE60	Copy	Dimension	DimStore	age_over_60_pct
stg_Store	WORKWOM	Copy	Dimension	DimStore	working_women_pct
stg_Product	(auto)	Generate surrogate key	Dimension	DimProduct	product_key (PK)
stg_Product	UPC	Copy	Dimension	DimProduct	upc
stg_Product	DESCRIP	Copy	Dimension	DimProduct	description
stg_Product	SIZE	Copy	Dimension	DimProduct	size
stg_Product	CASE_PACK	Copy	Dimension	DimProduct	case_pack
stg_Product	COM_CODE	Copy	Dimension	DimProduct	commodity_code
stg_Product	NITEM	Copy	Dimension	DimProduct	item_number
(Generated)	WEEK values	Calculate dates from base	Dimension	DimTime	time_key, week_id, etc.
(Derived)	CATEGORY_CODE	Map codes to full names	Dimension	DimCategory	category_key, code, name
(Derived)	SALE values	Map: NULL to No Promotion; B/C/S to type	Dimension	DimPromotion	promotion_key, deal_code
stg_Movement	(auto)	Generate surrogate key	Fact	FactWeeklySales	sales_fact_id (PK)
stg_Movement	UPC	Lookup product_key	Fact	FactWeeklySales	product_key (FK)
stg_Movement	STORE	Lookup store_key	Fact	FactWeeklySales	store_key (FK)
stg_Movement	WEEK	Lookup time_key	Fact	FactWeeklySales	time_key (FK)
stg_Movement	CATEGORY_CODE	Lookup category_key	Fact	FactWeeklySales	category_key (FK)
stg_Movement	SALE	Lookup	Fact	FactWeeklySales	promotion_key (FK)

		promotion_key			
stg_Movement	MOVE	Copy	Fact	FactWeeklySales	units_sold
stg_Movement	PRICE,QTY	Transform: PRICE/QTY	Fact	FactWeeklySales	unit_price
stg_Movement	PRICE	Copy	Fact	FactWeeklySales	shelf_price
stg_Movement	QTY	Copy	Fact	FactWeeklySales	price_qty
stg_Movement	MOVE,PRICE,QTY	MOVE x (PRICE/QTY)	Fact	FactWeeklySales	revenue
stg_Movement	PROFIT	Copy	Fact	FactWeeklySales	gross_profit
stg_Movement	PROFIT,revenue	(PROFIT/rev) x 100	Fact	FactWeeklySales	profit_margin_pct

5. Physical Design

The physical design transforms the logical star schema into a deployable structure on SQL Server 2016. Since DFF's data marts will grow as new product categories, stores, or time periods are added, each area is planned for scalability. With FactWeeklySales containing approximately 134.9 million rows at the UPC x Store x Week grain, pre-computed aggregate (summary) tables will improve query performance for frequently asked questions. Three aggregate tables are planned: agg_Weekly_Category_Sales (aggregating units_sold and revenue by week and category, speeding up BQ2-type queries), agg_Store_Category_Revenue (total revenue by store and category, speeding up BQ8 quartile analysis), and agg_Weekly_Product_Sales (units_sold by week and product per category, speeding up BQ9 rankings). For indexing, the fact table will use a clustered index on the composite key (time_key, store_key, product_key), with non-clustered indexes on promotion_key and category_key for filtered queries like BQ3 and BQ4. Dimension tables will use unique non-clustered indexes on surrogate primary keys. For columns with low cardinality such as DimPromotion.deal_code (4 values), DimStore.is_urban (2 values), and DimStore.price_tier (3 values), filtered non-clustered indexes will be created for each distinct value to speed up common query patterns on SQL Server 2016. A non-clustered columnstore index on FactWeeklySales will further accelerate analytical aggregations. During bulk ETL loads, indexes will be dropped before loading and recreated afterward to avoid slowing down the load process.

Naming standards ensure consistency across the warehouse: dimension tables are prefixed "Dim," fact tables "Fact," staging tables "stg_," and aggregate tables "agg_." All surrogate keys are 4-byte INT, monetary values use DECIMAL(8,2) or DECIMAL(12,2), descriptive text uses VARCHAR(n), and flags use BIT. Null handling rules are enforced during ETL: SALE values that are NULL or blank are mapped to the "No Promotion" row in DimPromotion (promotion_key = 1), DEMO.csv NAME/CITY nulls become "UNKNOWN," and Movement rows with OK = 0 or PRICE = 0 are excluded during staging. For storage, FactWeeklySales will be horizontally partitioned by time_key using yearly partitions (about 52 weeks each), ensuring queries only scan relevant partitions, new data loads do not lock historical data, and backup and recovery operates at the partition level. Estimated total storage: FactWeeklySales about 4 to 5 GB, dimension tables under 10 MB, aggregates under 50 MB, indexes about 30 to 50 percent of base table size, staging area about 2 GB temporary. The independent data mart architecture allows new data marts to be added without modifying the existing FactWeeklySales schema or its conformed dimensions.